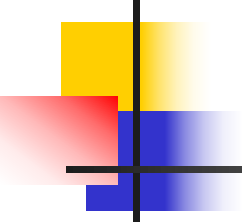


第4章 卷积神经网络及应用



湖北大学 计算机学院
王雷春



目 录

4.1 概述

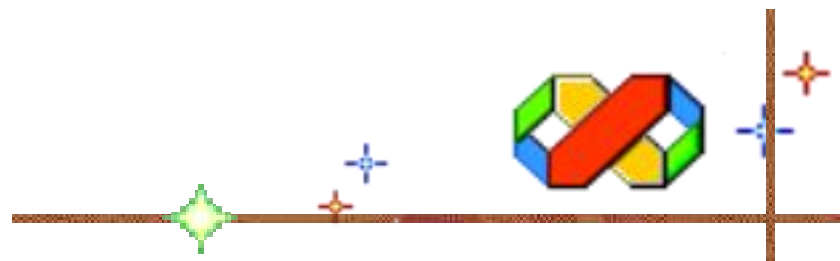
4.2 CNN分类

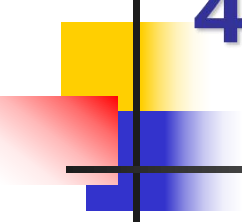
4.3 基础CNN

4.4 常见CNN

4.5 CNN应用

4.6 CNN局限性

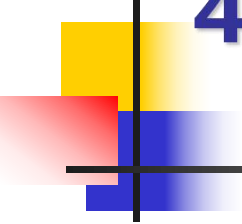




4.1 生物神经元

1. 背景与动机

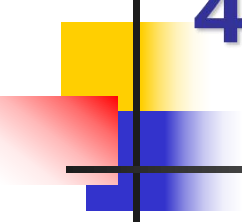
- ◆ (1)生物视觉研究的启发。20世纪60年代，Hubel和Wiesel发现猫的视觉皮层存在“简单细胞”和“复杂细胞”的层级结构，分别负责边缘检测和特征聚合。这一发现是CNN的“卷积-池化”结构的重要思想来源。
- ◆ (2)传统模式识别方法的局限。在CNN成熟之前，图像识别需要人工设计特征，使用浅层学习模型（如SVM、MLP等），导致设计繁琐复杂、模型效果差和学习参数爆炸等不足。
- ◆ (3)关键算法与技术的成熟。反向传播算法、时间延迟网络（TDNN）和卷积+池化操作的形式化等的出现为CNN的实现提供了技术基础。



4.1 生物神经元

2. 什么是CNN

- ◆ CNN是一类强大的、为处理网格状数据（如图像、视频等）设计的深度学习模型。
- ◆ CNN通常由卷积层、非线性激活函数、池化层和全连接层等组件堆叠而成，通过卷积核在输入上滑动计算互相关（工程中常简称为卷积），生成特征图，从而自动学习从低级边缘到高级语义的分层特征表示。
- ◆ 其中，C表示卷积，是这种网络的核心操作，用于提取各种特征（如边缘、纹理）；NN表示神经网络，是一种受人脑神经元连接方式启发而设计的计算模型。



4.1 生物神经元

2. 什么是CNN

形象化比喻

- ◆ 不同科室的专科医生在CT影像上滑动寻找关心的病灶，并根据该科室的标准确定是否达到规定阈值。
- ◆ 各个科室主任将各个专科医生汇总的疑似病灶进行分析、判断，只保留最关键的内容。
- ◆ 最后，会诊团队对各个科室提供的信息进行综合会诊，得出出结论。
- ◆ 这里，CT影像对应要进行分析的图像，专科医生对应CNN的卷积核，病灶上报标准对应非线性激活函数，科室主任对应CNN的池化层，多学科联合会诊对应全连接层，会诊结果对应输出层。

4.1 生物神经元

3. 设计思想

- ◆ (1)局部连接。在图像中，邻近像素相关性高，远距离像素相关性弱。CNN不再像全连接网络那样将每个神经元与上一层所有神经元相连，而是每个神经元仅连接输入的一小块区域（感受野），而非全部。通过这种方式，极大地减少了参数数量。在卷积核尺寸通常为 3×3 、 5×5 ，远小于输入尺寸。
- ◆ (2)权值共享。图像中某一特征（如边缘、角点）可能出现在任何位置，无需为每个位置单独学习检测器。同一个卷积核（滤波器）在输入图像上滑动时，其权重参数保持不变。因此，无论局部特征出现在图像哪个位置，都会被同一组参数检测到。这不但降低了模型参数量，还赋予了模型平移不变性。

4.1 生物神经元

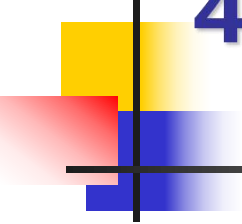
3. 设计思想

- ◆ (3)层次化特征学习。通过堆叠多个卷积层，网络能自动学习从低级到高级的特征：
 - ◆ 浅层 → 边缘、颜色、纹理
 - ◆ 中层 → 形状、部件（眼睛、轮子）
 - ◆ 深层 → 整体语义（人脸、汽车）
- ◆ (4)池化（下采样）。在卷积层之后加入池化层，对局部区域进行聚合统计，以降低特征图分辨率、减少计算量、增强对微小平移和形变的鲁棒性，同时扩大后续层的感受野。

4.1 生物神经元

4. CNN特点

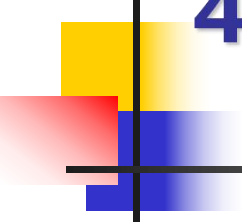
- ◆ (1) 参数数量大幅减少。通过局部连接和权值共享，CNN的参数总量远低于全连接网络。
 - ◆ 例如，处理一张 100×100 的图像，全连接层需要千万级参数，而CNN只需几十到几百个卷积核参数。
- ◆ (2) 自动提取层次化特征。无需人工设计特征，而是从原始像素中端到端学习从边缘、纹理到物体部件、类别的高层语义特征。
- ◆ (3) 平移不变性。由于权值共享和池化操作，CNN对输入中物体位置变化不敏感，识别更稳定。



4.1 生物神经元

4. CNN特点

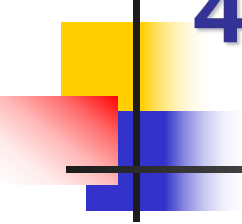
- ◆ (4)计算高效。由于权值共享和局部连接，卷积运算可以通过快速算法（如 $\text{im2col} + \text{GEMM}$ 、FFT、Winograd）高效实现，并且非常适合 GPU 并行加速。相比全连接网络，训练和推理速度更快。
- ◆ (5)泛化能力强。对图像中的局部畸变、简单形变（如轻微旋转、拉伸）有一定容忍度，在视觉任务中往往优于传统方法。
- ◆ (6)适用广泛。CNN不仅用于图像分类、目标检测、语义分割，还拓展到视频分析、语音识别、自然语言处理（文本序列的一维卷积）等领域。



4.2 CNN分类

1. 按数据维度划分

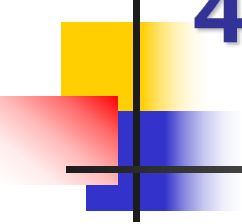
- ◆ (1) 1D CNN(一维卷积): 卷积核在数据上沿一个方向 (通常是时间) 滑动。擅长处理时间序列数据, 如传感器信号、心电图、音频信号或简单的文本序列。
- ◆ (2) 2D CNN(二维卷积): 最经典的CNN类型, 卷积核在高度和宽度两个维度上滑动。处理图像数据的绝对主力, 几乎所有图像分类、目标检测和分割任务都基于此。
- ◆ (3) 3D CNN(三维卷积): 卷积核在三个维度上 (通常是图像的宽、高和时间) 滑动。能够同时捕捉空间特征和短期时间动态, 非常适合视频分析、行为识别和3D医学影像 (如CT、MRI) 处理。



4.2 CNN分类

1. 按数据维度划分

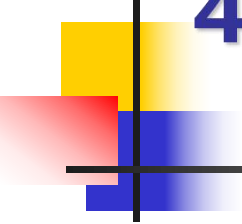
- ◆ (4)时空卷积网络：是3D CNN的一种演变，旨在更高效地建模时空数据。采用组合方式，例如将空间（2D）和时间（1D）卷积分离成两个步骤，以平衡精度和计算量，这类模型（如R(2+1)D、I3D、P3D）在视频理解任务中非常流行。



4.2 CNN分类

2. 按架构演进划分

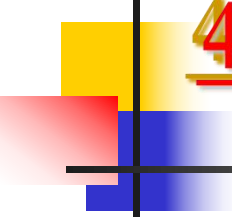
- ◆ (1)LeNet-5: 由Yann LeCun在1998年提出, 是CNN的开山之作, 首次成功将卷积、池化和全连接层组合成端到端的可训练网络。
- ◆ (2)AlexNet: 由Alex Krizhevsky在2012年提出, 引爆深度学习革命, 在ImageNet 2012竞赛中以绝对优势夺冠。
- ◆ (3)VGGNet: 由牛津大学VGG组在2014年提出, 验证了网络深度对性能的重要性, 结构极其规整。
- ◆ (4)GoogLeNet/Inception-v1: 由Google在2014年提出), 在加深网络的同时控制计算量, 赢得2014年ImageNet冠军。



4.2 CNN分类

2. 按架构演进划分

- ◆ (5)ResNet: 由微软亚洲研究院的何恺明团队在2015年提出, 解决极深网络退化问题 (层数增多但精度反而下降), 使训练上百乃至上千层网络成为可能。
- ◆ (6)DenseNet: 由康奈尔大学的黄高团队在2017年提出, 进一步极致化特征重用, 缓解梯度消失并提高参数效率。



4.3 基础CNN

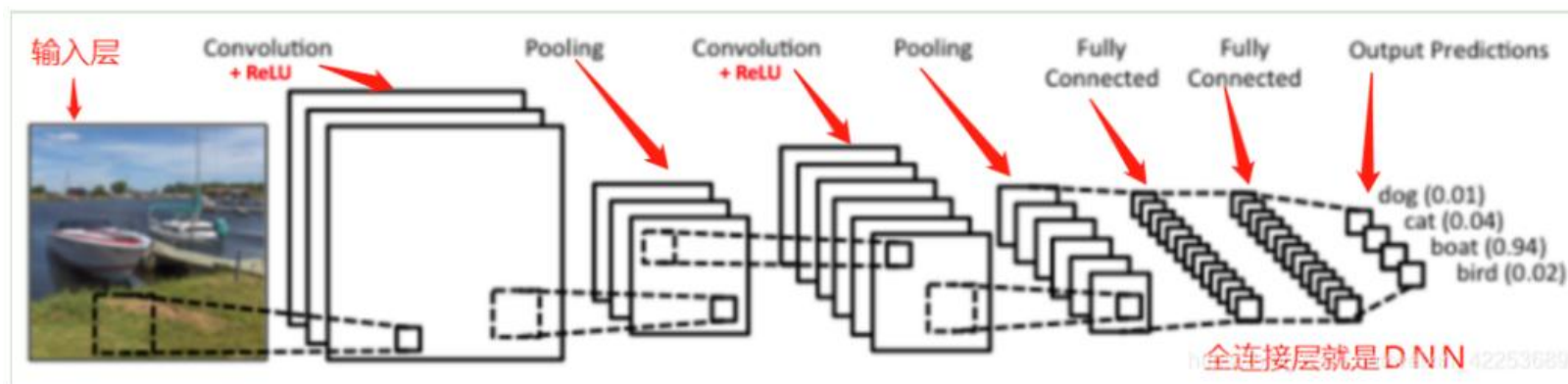
4.3.1 基础CNN结构

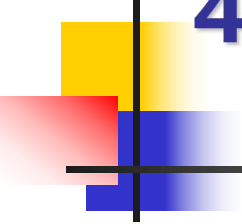
4.3.2 案例



4.3.1 基础CNN结构

- ◆ 经典CNN的核心结构由输入层、卷积层、激活函数、池化层、全连接层和输出层依次堆叠而成（如图所示）。



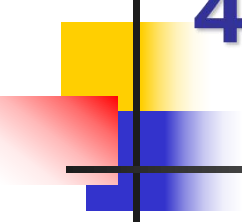


4.3.1 基础CNN结构

1. 输入层

1. 输入层

- ◆ 输入层的作用是接收原始数据（如图像的像素矩阵）。
- ◆ 图像为3维张量：彩色图像的形状为 (高度, 宽度, 通道数)，灰度图像的形状为 (H, W, 1)。



4.3.1 基础CNN结构

2. 卷积层

2. 卷积层

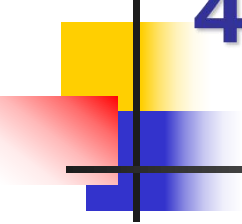
- ◆ 卷积层是CNN的核心，其作用是提取图像的核心特征提取。每个卷积核负责提取一种特定的局部特征，如边缘、纹理、颜色等。浅层提取简单特征，深层组合成复杂特征（如眼睛、车轮）。
- ◆ 核心操作：使用多个可学习的卷积核（也叫滤波器）在输入数据上滑动，进行互相关运算（实践中常简称为卷积运算），生成特征图。

4.3.1 基础CNN结构

2. 卷积层

◆ 关键参数：

- ◆ 核大小 (kernel size) : 常见 1×1 , 3×3 , 5×5 。
- ◆ 步长 (stride) : 核滑动的步长, 通常为1或2。
- ◆ 填充 (padding) : 在输入边界补零, 用于控制输出尺寸 (保持或减小)。常用 “same” 填充 (输出与输入尺寸相同) 或 “valid” 填充 (不填充, 输出缩小) 。
- ◆ 输出通道数: 等于本层使用的卷积核个数 (每个核提取一种特征) 。



4.3.1 基础CNN结构

3. 激活函数

3. 激活函数

- ◆ 激活函数的作用是在卷积操作后加入非线性变换，提高网络的模型逼近能力。
- ◆ CNN中常用激活函数为ReLU 及其变种（Leaky ReLU、ELU等）。

4.3.1 基础CNN结构

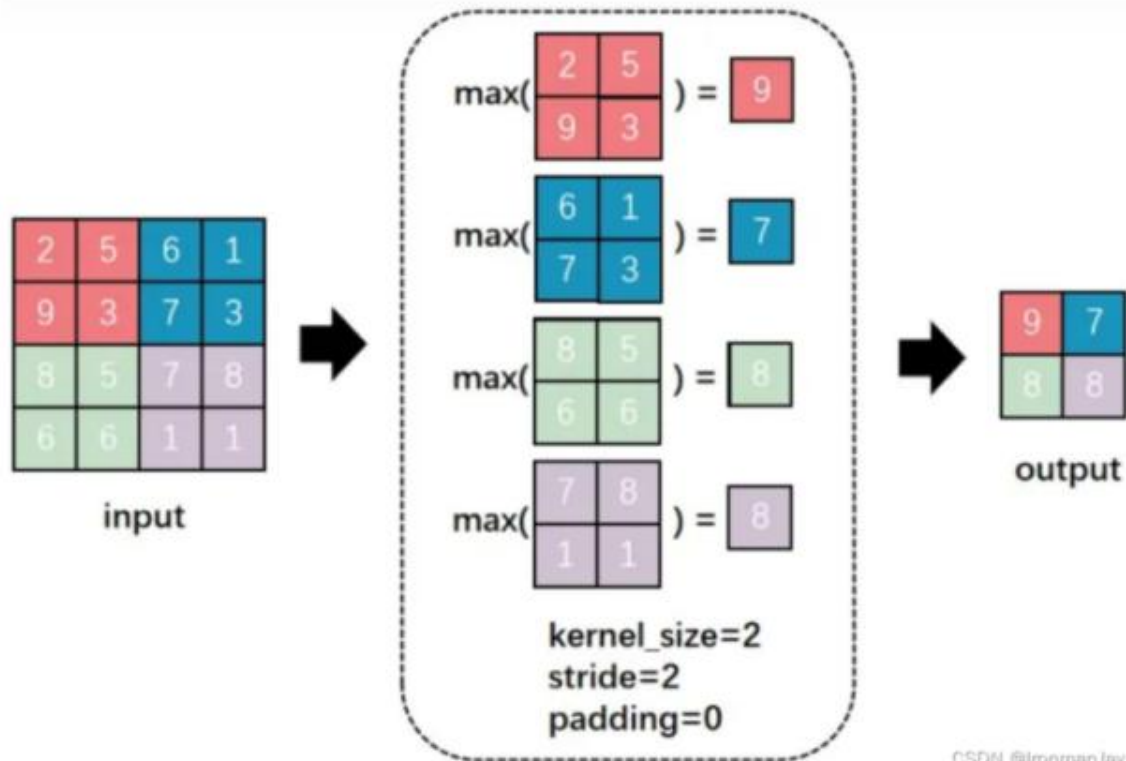
4. 池化层

4. 池化层

- ◆ 池化层的作用是对每个特征图进行局部聚合，降低特征图的空间尺寸（宽和高），减少计算量，同时增强特征的平移不变性（即使物体微移，池化后的结果变化不大）。
- ◆ 最大池化：取窗口内最大值，保留最显著特征，常用 2×2 步长2，输出尺寸减半。
- ◆ 平均池化：取窗口内均值，保留背景信息。
- ◆ 全局平均池化（Global Average Pooling）：将每个通道的特征图缩放到一个值，直接输出通道数个特征，替代全连接层，大幅减少参数。

4.3.1 基础CNN结构

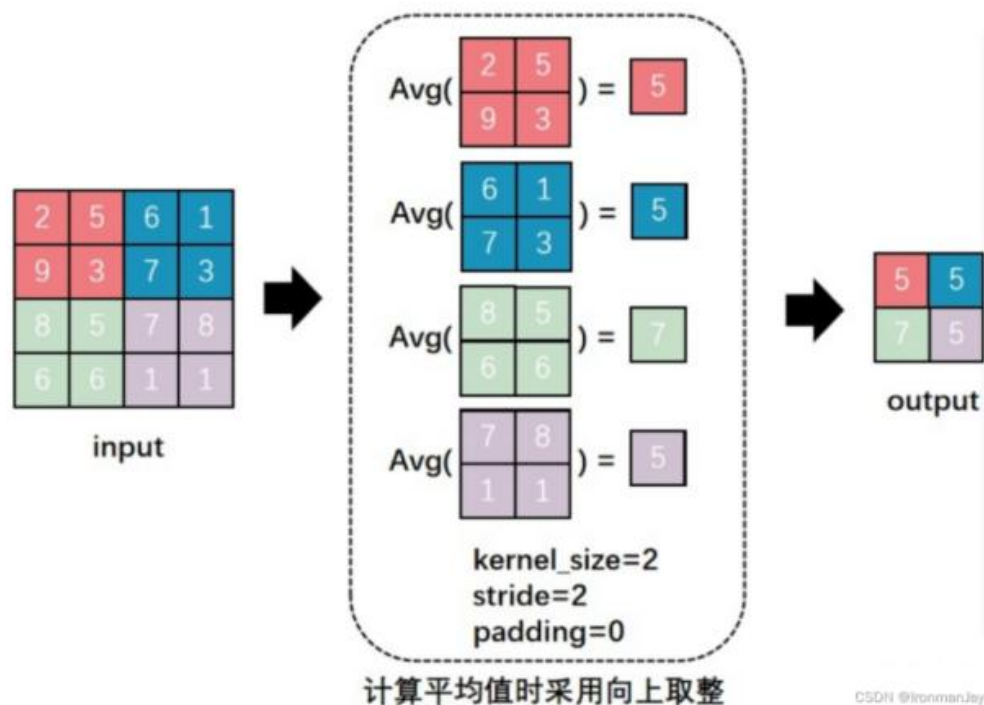
4. 池化层



CSDN @IronmanJey

4.3.1 基础CNN结构

4. 池化层



4.3.1 基础CNN结构

5. 全连接层

5. 全连接层

- ◆ 作用：位于网络末端，将卷积池化提取到的高阶特征展平后，进行非线性组合，完成最终的分类或回归任务。
- ◆ 特点：参数数量巨大（约占网络总参数的80%以上），容易过拟合，现代网络中常被全局平均池化层部分替代。

4.3.1 基础CNN结构

6. 输出层

6. 输出层

- ◆ 作用：根据任务输出最终结果。
- ◆ 常见配置：
 - ◆ 分类任务：全连接层 + Softmax函数（输出类别概率）。
 - ◆ 回归任务：全连接层 + 线性激活（如预测坐标、深度）。

4.3.1 基础CNN结构

案 例

【例4-2】 设计一个用于灰度图像分类的卷积神经网络（CNN）。输入为单通道（ $1 \times H \times W$ ），输出为类别数。

- ◆ (1)模型设计
- ◆ 模型结构参考经典CNN结构，兼顾简洁性和性能，包括2个卷积块和1个全连接分类器：
- ◆ 卷积块 1/2
 - ◆ Conv2d(1, 32/64, kernel_size=3, padding=1)
 - ◆ BatchNorm2d(32/64)
 - ◆ ReLU
 - ◆ MaxPool2d(2, stride=2) → 尺寸减半

4.3.1 基础CNN结构

案 例

- ◆ 全连接分类器
- ◆ 自适应平均池化 `AdaptiveAvgPool2d((4,4))` 将特征图统一到固定大小
 - ◆ Flatten
 - ◆ `Linear(128 * 4 * 4, 256)`
 - ◆ ReLU
 - ◆ `Dropout(0.5)`
 - ◆ `Linear(256, num_classes)`

4.3.1 基础CNN结构

案例

程序代码

```
◆ import torch.nn as nn
◆ class GrayImageCNN(nn.Module):
◆     def __init__(self, num_classes=10, input_channels=1, image_size=28):
◆         super(GrayImageCNN, self).__init__()
◆         # 卷积层
◆         self.conv_block1 = nn.Sequential(
◆             nn.Conv2d(input_channels, 32, kernel_size=3, padding=1),
◆             nn.BatchNorm2d(32),
◆             nn.ReLU(inplace=True),
◆             nn.MaxPool2d(kernel_size=2, stride=2)
◆         )
```

4.3.1 基础CNN结构

案例

程序代码

```
◆ self.conv_block2 = nn.Sequential(  
◆     nn.Conv2d(32, 64, kernel_size=3, padding=1),  
◆     nn.BatchNorm2d(64),  
◆     nn.ReLU(inplace=True),  
◆     nn.MaxPool2d(kernel_size=2, stride=2)  
◆ )
```

4.3.1 基础CNN结构

案例

程序代码

- ◆ # 使用自适应池化将特征图统一到固定大小 (比如 4x4)
- ◆ `self.adaptive_pool = nn.AdaptiveAvgPool2d((4, 4))`
- ◆ # 计算全连接层输入维度, 最后一个卷积输出通道 64, 特征图 4x4, 所以 $64 * 4 * 4$
- ◆ `self.fc = nn.Sequential(`
- ◆ `nn.Flatten(),`
- ◆ `nn.Linear(64 * 4 * 4, 256),`
- ◆ `nn.ReLU(inplace=True),`
- ◆ `nn.Dropout(0.5),`
- ◆ `nn.Linear(256, num_classes)`

4.3.1 基础CNN结构

案 例

程序代码

- ◆ def forward(self, x):
- ◆ x = self.conv_block1(x)
- ◆ x = self.conv_block2(x)
- ◆ x = self.adaptive_pool(x)
- ◆ x = self.fc(x)
- ◆ return x

- ◆ model_cnn = GrayImageCNN()
- ◆ print(model_cnn)

4.3.1 基础CNN结构

案 例

运行结果

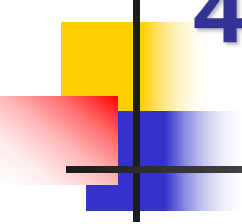
```
GrayImageCNN(  
  (conv_block1): Sequential(  
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (2): ReLU(inplace=True)  
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (conv_block2): Sequential(  
    (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    (2): ReLU(inplace=True)  
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
  )  
  (adaptive_pool): AdaptiveAvgPool2d(output_size=(4, 4))
```

4.3.1 基础CNN结构

案 例

运行结果

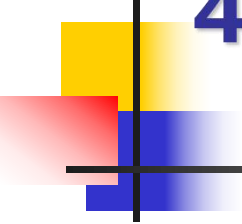
```
(fc): Sequential(  
  (0): Flatten(start_dim=1, end_dim=-1)  
  (1): Linear(in_features=1024, out_features=256, bias=True)  
  (2): ReLU(inplace=True)  
  (3): Dropout(p=0.5, inplace=False)  
  (4): Linear(in_features=256, out_features=10, bias=True)  
)  
)
```

4.3.2 案例

1. CIFAR-10

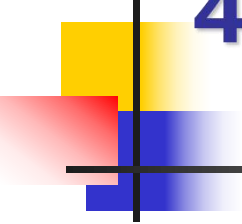
- ◆ CIFAR-10是一个在计算机视觉领域广泛使用的图像分类数据集,由加拿大高级研究所 (CIFAR) 创建。
- ◆ CIFAR-10规模适中 (比MNIST复杂, 比ImageNet简单) , 是测试新图像分类算法的理想基准。
- ◆ CIFAR-10分为10个类别, 分别是飞机 (plane)、汽车 (car)、鸟 (bird)、猫 (cat)、鹿 (deer)、狗 (dog)、青蛙 (frog)、马 (horse)、船 (ship)、卡车 (truck) 。



4.3.2 案例

1. CIFAR-10

- ◆ CIFAR-10包含60000张32x32彩色图像。
- ◆ 训练图50000张，测试图10000张，每类训练和测试图片数量相等（各5000和1000张）。
- ◆ 每张图像都是32x32像素的3通道小尺寸彩色图，即便放大后，人类也难以仅凭肉眼准确识别所有细节。
- ◆ 某些类别（如猫和狗、汽车和卡车）之间存在较高的类间相似性，使精确分类颇具挑战。
- ◆ 由于CIFAR-10的图像尺寸较小（32×32），直接使用在ImageNet上预训练的CNN模型存在尺寸不匹配的问题——大多数预训练模型（如ResNet、VGG）要求输入尺寸为224×224。



4.3.2 案例

2. 步骤

(1)数据加载

- ◆ 程序运行时会自动检查需要的数据集CIFAR-10是否存在：如果不存在，自动下载。建议先下载好数据集CIFAR-10，放在指定的目录，以加快程序的运行。
- ◆ 然后，对数据集CIFAR-10进行预处理，包括归一化和标准化。

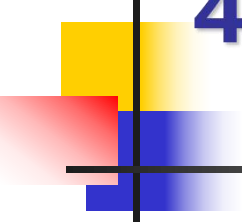
4.3.2 案例

2. 步骤

(2)模型定义

◆ CNN 模型结构定义如下：

- ◆ 三个卷积层，其中`kernel_size=3`, `padding=1`。每个卷积层后跟 ReLU 和 2x2 最大池化。
- ◆ 三个全连接层，中间加入 Dropout 防止过拟合。
- ◆ 输出 10 个类别得分。
- ◆ 损失函数采用 CrossEntropyLoss 。
- ◆ 优化器采用 Adam，学习率初始值设为 0.001。



4.3.2 案例

2. 步骤

(3)训练

- ◆ 将训练过程封装在train()函数中，具体训练时进行调用即可。训练轮数设为10。

(4)测试

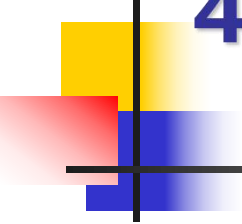
- ◆ 将训练过程封装在train()函数中，具体训练时进行调用即可。

(5)保存模型

- ◆ 在测试集上评估准确率，并将模型参数保存为 .pth 文件。

(6)新图片预测

- ◆ 载保存的模型，对测试集中的一张图片进行预测，并与真实标签对比。



4.3.2 案例

3. 程序代码

- ◆ 具体代码参见和教材配套的代码文件。

4.3.2 案例

4. 程运行结果

- ◆ Epoch 1 finished, Training Accuracy: 41.87%
- ◆ Epoch 2 finished, Training Accuracy: 59.03%
- ◆ ...
- ◆ Epoch 10 finished, Training Accuracy: **81.37%**
- ◆ Finished Training
- ◆ Test Accuracy: **75.82%**
- ◆ Model saved as 'cifar10_cnn.pth'
- ◆ True label: ***cat***
- ◆ Predicted label: ***cat***



4.3.2 案例

5. 分析和改进

- ◆ 【例4-3】中的模型测试的准确率一般只能达到70%左右，预测结果出错的概率比较高。如果想获得更高准确率，可以采用一些方法进行增强模型的泛化能力，从而提高预测结果的准确率。

运行结果：

- ◆ Epoch 1 | Loss: 1.5857 | Train Acc: 40.93% | Test Acc: 53.32%
- ◆ Epoch 2 | Loss: 1.1356 | Train Acc: 59.04% | Test Acc: 61.26%
- ◆
- ◆ Epoch 50 | Loss: 0.1314 | Train Acc: 95.64% | Test Acc: **89.24%**
- ◆ Training finished. Best test accuracy: **90.30%**
- ◆ Final test accuracy: **90.30%**

4.3.2 案例

5. 分析和改进

- ◆ (2)批量归一化 (Batch Normalization, 简称 BN) 。批量归一化是深度学习中一项非常重要的技术, 主要用于解决深层神经网络训练过程中的内部协变量偏移问题, 并显著加速训练、提高稳定性。本案例在模型中对图像特征数据上进行了批量归一化。
- ◆ `nn.BatchNorm2d(64)`
- ◆ (3)权重衰减。在优化器中添加Weight Decay(权重衰减)防止模型过拟合, 提升泛化能力。
- ◆ 本案例在优化器中添加 `weight_decay` 参数, 是实现 L2 正则化 (权重衰减) 的标准做法, 用于防止模型过拟合、提升泛化能力。
- ◆ `optimizer = optim.AdamW(model.parameters(), lr=0.001, weight_decay=5e-4)`

4.3.2 案例

5. 分析和改进

- ◆ 【例4-3】中的模型测试的准确率一般只能达到70%左右，预测结果出错的概率比较高。如果想获得更高准确率，可以采用一些方法进行增强模型的泛化能力，从而提高预测结果的准确率。

(1)数据增强。

- ◆ 几何变换：旋转、平移、缩放、翻转等（本质是模拟相机角度和距离的变化）；
- ◆ 色彩变换：调整亮度、对比度、饱和度（本质是模拟不同光照条件）；
- ◆ 添加噪声：高斯噪声、椒盐噪声（本质是模拟传感器或传输中的干扰）。

- ◆ (2)批量归一化（Batch Normalization，简称 BN）。批量归一化是深度学习